

LINUX MULTITHREADING

Multithreading - Printing Odd and Even Numbers Using Two Threads

1. Write a C program using POSIX threads where one thread prints odd numbers and another thread prints even numbers up to 100, ensuring the output alternates correctly.

PROGRAM:

```
/*
Program to print odd and even numbers from 1 to 100 using
mutex and condition variables.
Author   : MUTHUGANESH S
Date     : 11/03/2026
Filename : oe.c
*/

// Include necessary headers
#include <stdio.h>
#include <pthread.h>

// Declare mutex and condition variable
pthread_mutex_t lock;
pthread_cond_t cond;

// Global variable to track turn
int turn = 1; // 1 = odd thread, 0 = even thread

// Function to print odd numbers
void* print_odd(void* arg) {
    for (int i = 1; i <= 100; i += 2) {
        pthread_mutex_lock(&lock);

        while (turn != 1)
            pthread_cond_wait(&cond, &lock);

        printf("%d ", i);

        turn = 0;
        pthread_cond_signal(&cond);

        pthread_mutex_unlock(&lock);
    }
    return NULL;
}

// Function to print even numbers
```

```

void* print_even(void* arg) {
    for (int i = 2; i <= 100; i += 2) {
        pthread_mutex_lock(&lock);

        while (turn != 0)
            pthread_cond_wait(&cond, &lock);

        printf("%d ", i);

        turn = 1;
        pthread_cond_signal(&cond);

        pthread_mutex_unlock(&lock);
    }
    return NULL;
}

int main() {
    pthread_t t1, t2;

    pthread_mutex_init(&lock, NULL);
    pthread_cond_init(&cond, NULL);

    pthread_create(&t1, NULL, print_odd, NULL);
    pthread_create(&t2, NULL, print_even, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    pthread_mutex_destroy(&lock);
    pthread_cond_destroy(&cond);

    return 0;
}

```

OUTPUT:

```

mg@MG:/mnt/m/training/LINUX/MULTI THREADING$ gcc oe.c -o oe
mg@MG:/mnt/m/training/LINUX/MULTI THREADING$ ./oe
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43
44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 8
3 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100 mg@MG:/mnt/m/training/LINUX/MULTI THREADING$

```

2. Implement a program with two threads: one prints odd numbers and the other prints even numbers. Use synchronization primitives (mutexes or condition variables) to avoid race conditions and ensure correct sequence..

PROGRAM:

```
/*
Program to print odd and even numbers from 1 to 100 using
mutex.
Author   : MUTHUGANESH S
Date     : 11/03/2026
Filename : ooe.c
*/

// This program creates two threads: one for printing odd
numbers and another for printing even numbers.
#include <stdio.h>
#include <pthread.h>

// Mutex to synchronize access to the shared variable 'turn'.
pthread_mutex_t lock;
int turn = 1;

// Function to print odd numbers from 1 to 100.
void* print_odd(void* arg) {
    for (int i = 1; i <= 100; i += 2) {
        while (1) {
            pthread_mutex_lock(&lock);

            if (turn == 1) {
                printf("%d ", i);
                turn = 0;
                pthread_mutex_unlock(&lock);
                break;
            }

            pthread_mutex_unlock(&lock);
        }
    }
    return NULL;
}

// Function to print even numbers from 1 to 100.
void* print_even(void* arg) {
    for (int i = 2; i <= 100; i += 2) {
        while (1) {
            pthread_mutex_lock(&lock);
```

```

        if (turn == 0) {
            printf("%d ", i);
            turn = 1;
            pthread_mutex_unlock(&lock);
            break;
        }

        pthread_mutex_unlock(&lock);
    }
}
return NULL;
}

int main() {
    pthread_t t1, t2;

    pthread_mutex_init(&lock, NULL);

    pthread_create(&t1, NULL, print_odd, NULL);
    pthread_create(&t2, NULL, print_even, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    pthread_mutex_destroy(&lock);

    return 0;
}

```

OUTPUT:

```

mg@MG:/mnt/m/training/LINUX/MULTI THREADING$ gcc ooe.c -o ooe
mg@MG:/mnt/m/training/LINUX/MULTI THREADING$ ./ooe
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43
44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 8
3 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100 mg@MG:/mnt/m/training/LINUX/MULTI THREADING$ |

```

pthread_cancel

1. Write a C program that creates a thread and cancels it from the main thread using `pthread_cancel()`. Demonstrate how the thread responds to cancellation.

PROGRAM:

```

/*
Program to demonstrate thread cancellation
Author   : MUTHUGANESH S
Date     : 11/03/2026
Filename : can.c
*/

```

```

#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

void* worker(void* arg) {
    while (1) {
        printf("Worker thread running...\n");
        sleep(1); // cancellation point
    }
    return NULL;
}

int main() {
    pthread_t tid;

    pthread_create(&tid, NULL, worker, NULL);

    sleep(3); // let thread run

    printf("Main thread: Cancelling worker...\n");
    pthread_cancel(tid);

    void* status;
    pthread_join(tid, &status);

    if (status == PTHREAD_CANCELED)
        printf("Worker thread was cancelled successfully.\n");
    else
        printf("Worker thread exited normally.\n");

    return 0;
}

```

OUTPUT:

```

mg@MG:/mnt/m/training/LINUX/MULTI THREADING$ gcc can.c -o can
mg@MG:/mnt/m/training/LINUX/MULTI THREADING$ ./can
Worker thread running...
Worker thread running...
Worker thread running...
Main thread: Cancelling worker...
Worker thread running...
Worker thread was cancelled successfully.
mg@MG:/mnt/m/training/LINUX/MULTI THREADING$ |

```

2. Implement a program where a worker thread performs a long computation. The main thread cancels the worker after a timeout and checks the cancellation status.

PROGRAM:

```
/*
Program to demonstrate controlled thread cancellation
Author   : MUTHUGANESH S
Date     : 11/03/2026
Filename : con.c
*/

#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

// Worker thread function
void* worker(void* arg) {
    long long i = 0;

    while (1) {
        i++;

        // simulate heavy computation
        if (i % 10000000 == 0) {
            printf("Working... i = %lld\t", i);

            // explicit cancellation point
            pthread_testcancel();
        }
    }

    return NULL;
}

int main() {
    pthread_t tid;

    pthread_create(&tid, NULL, worker, NULL);

    sleep(2); // allow computation

    printf("Main: Requesting cancellation...\n");
    pthread_cancel(tid);

    void* status;
    pthread_join(tid, &status);

    if (status == PTHREAD_CANCELED)
```

OUTPUT:

pthread_detach

1. Write a C program that creates a thread and detaches it using pthread_detach(). Show what happens if the main thread exits before the detached thread finishes.

PROGRAM:

```
/*
Program to demonstrate the use of pthread_detach in C.
Author   : MUTHUGANESH S
Date     : 11/03/2026
Filename : det.c
*/

//Header files
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

// Worker thread function
void* worker(void* arg) {
    for (int i = 1; i <= 5; i++) {
        printf("Worker running: %d\n", i);
        sleep(1);
    }
    printf("Worker finished\n");
    return NULL;
}

int main() {
    pthread_t tid;

    pthread_create(&tid, NULL, worker, NULL);

    pthread_detach(tid);

    printf("Main thread exiting immediately...\n");

    return 0;
}
```

OUTPUT:

```
mg@MG:/mnt/m/training/LINUX/MULTI THREADING$ gcc det.c -o det
mg@MG:/mnt/m/training/LINUX/MULTI THREADING$ ./det
Main thread exiting immediately...
mg@MG:/mnt/m/training/LINUX/MULTI THREADING$
```


2. Implement a program where multiple threads are created and detached. Demonstrate that their resources are automatically reclaimed upon termination.

PROGRAM:

```
/*
Program to demonstrate the use of pthread_join in C.
Author   : MUTHUGANESH S
Date     : 11/03/2026
Filename : join.c
*/

//Header files
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

// Number of threads to create
#define N 5

// Worker thread function
void* worker(void* arg) {
    int id = *(int*)arg;
    printf("Thread %d started\n", id);

    sleep(2);

    printf("Thread %d finished\n", id);
    return NULL;
}

int main() {
    pthread_t tid[N];
    int ids[N];

    for (int i = 0; i < N; i++) {
        ids[i] = i + 1;

        pthread_create(&tid[i], NULL, worker, &ids[i]);

        pthread_detach(tid[i]); // detach immediately
    }

    printf("Main thread will not join threads\n");

    // Keep process alive long enough
    sleep(3);

    printf("Main exiting\n");
}
```

```
    return 0;
}
```

OUTPUT:

```
main thread exiting immediately...
mg@MG:/mnt/m/training/LINUX/MULTI THREADING$ gcc join.c -o join
mg@MG:/mnt/m/training/LINUX/MULTI THREADING$ ./join
Thread 1 started
Thread 2 started
Thread 3 started
Thread 4 started
Main thread will not join threads
Thread 5 started
Thread 2 finished
Thread 1 finished
Thread 3 finished
Thread 4 finished
Thread 5 finished
Main exiting
mg@MG:/mnt/m/training/LINUX/MULTI THREADING$
```

pthread_join

1. Write a C program that creates several threads and uses pthread_join() to wait for each to finish, collecting their return values.

PROGRAM:

```
/*
Program to demonstrate multi join.
Author   : MUTHUGANESH S
Date     : 11/03/2026
Filename : mj.c
*/

// Header files
#include <stdio.h>
#include <pthread.h>
#include <stdlib.h>

// Number of threads to create
#define N 5

// Worker thread function
void* worker(void* arg) {
    int id = *(int*)arg;

    int* result = malloc(sizeof(int));
    *result = id * id; // example computation
}
```

```

    printf("Thread %d computed %d\n", id, *result);

    return result;
}

int main() {
    // Array to hold thread IDs and their corresponding
    arguments
    pthread_t tid[N];
    int ids[N];

    for (int i = 0; i < N; i++) {
        ids[i] = i + 1;
        pthread_create(&tid[i], NULL, worker, &ids[i]);
    }

    for (int i = 0; i < N; i++) {
        void* ret;
        pthread_join(tid[i], &ret);

        int* value = (int*)ret;
        printf("Joined thread %d, result = %d\n", i + 1,
*value);

        free(value); // critical to avoid memory leak
    }

    return 0;
}

```

OUTPUT:

```

mg@MG:/mnt/m/training/LINUX/MULTI THREADING$ gcc mj.c -o mj
mg@MG:/mnt/m/training/LINUX/MULTI THREADING$ ./mj
Thread 1 computed 1
Thread 2 computed 4
Thread 3 computed 9
Thread 4 computed 16
Joined thread 1, result = 1
Joined thread 2, result = 4
Joined thread 3, result = 9
Joined thread 4, result = 16
Thread 5 computed 25
Joined thread 5, result = 25
mg@MG:/mnt/m/training/LINUX/MULTI THREADING$

```

2. Implement a program where the main thread waits for a worker thread to complete using `pthread_join()`, and prints the result returned by the worker.

PROGRAM:

```
/*
Program to show single thread returning a value to main thread
using pthread_join() function.
Author   : MUTHUGANESH S
Date     : 11/03/2026
Filename : sj.c
*/

// Header files
#include <stdio.h>
#include <pthread.h>
#include <stdlib.h>

// Worker thread function
void* worker(void* arg) {
    // Cast the argument to an integer pointer and dereference
    it
    int n = *(int*)arg;

    // Perform some computation (for demonstration, we'll just
    double the input)
    int* result = malloc(sizeof(int));
    *result = n * 2;

    printf("Worker processed value\n");

    return result;
}

int main() {
    // Declare a thread ID and an input value for the worker
    thread
    pthread_t tid;
    int input = 10;

    pthread_create(&tid, NULL, worker, &input);

    void* ret;
    pthread_join(tid, &ret);

    int* value = (int*)ret;
    printf("Result from thread = %d\n", *value);

    free(value);
}
```

```
    return 0;
}
```

OUTPUT:

```
mg@MG:/mnt/m/training/LINUX/MULTI THREADING$ gcc sj.c -o sj
mg@MG:/mnt/m/training/LINUX/MULTI THREADING$ ./sj
Worker processed value
Result from thread = 20
mg@MG:/mnt/m/training/LINUX/MULTI THREADING$
```

pthread_exit

1. Write a C program where a worker thread uses pthread_exit() to return a value to the main thread. The main thread should retrieve and print this value using pthread_join().

PROGRAM:

```
/*
Program to demonstrate pthread_exit() and returning values
from threads in C.
Author   : MUTHUGANESH S
Date     : 11/03/2026
Filename : exit.c
*/

// Header files
#include <stdio.h>
#include <pthread.h>
#include <stdlib.h>

// Worker thread function
void* worker(void* arg) {
    int x = *(int*)arg;

    int* result = malloc(sizeof(int));
    if (!result) pthread_exit(NULL);

    *result = x * 5;

    printf("Worker computed: %d\n", *result);

    pthread_exit(result);
}
```

```

int main() {
    pthread_t tid;
    int input = 7;

    pthread_create(&tid, NULL, worker, &input);

    void* ret;
    pthread_join(tid, &ret);

    if (ret != NULL) {
        int* value = (int*)ret;
        printf("Main received: %d\n", *value);
        free(value);
    } else {
        printf("Worker returned NULL\n");
    }

    return 0;
}

```

OUTPUT:

```

Result from thread = 20
mg@MG:/mnt/m/training/LINUX/MULTI THREADING$ gcc exit.c -o exit
mg@MG:/mnt/m/training/LINUX/MULTI THREADING$ ./exit
Worker computed: 35
Main received: 35
mg@MG:/mnt/m/training/LINUX/MULTI THREADING$ |

```

2. Implement a program that creates multiple threads, each calling `pthread_exit()` with a different exit status. The main thread should collect and display the exit status of each thread.

PROGRAM:

```

/*
Program to demonstrate multi pthread_exit() and returning
values from threads in C.
Author   : MUTHUGANESH S
Date     : 11/03/2026
Filename : multiexit.c
*/

// Header files
#include <stdio.h>
#include <pthread.h>
#include <stdlib.h>

```

```

// Number of threads to create
#define N 4

// Worker thread function
void* worker(void* arg) {
    int id = *(int*)arg;

    int* status = malloc(sizeof(int));
    if (!status) pthread_exit(NULL);

    *status = id * 10;

    printf("Thread %d exiting with %d\n", id, *status);

    pthread_exit(status);
}

int main() {
    pthread_t tid[N];
    int ids[N];

    for (int i = 0; i < N; i++) {
        ids[i] = i + 1;
        pthread_create(&tid[i], NULL, worker, &ids[i]);
    }

    for (int i = 0; i < N; i++) {
        void* ret;
        pthread_join(tid[i], &ret);

        if (ret != NULL) {
            int* value = (int*)ret;
            printf("Main collected from thread %d: %d\n", i +
1, *value);
            free(value);
        } else {
            printf("Thread %d returned NULL\n", i + 1);
        }
    }

    return 0;
}

```

OUTPUT:

```
mg@MG:/mnt/m/training/LINUX/MULTI THREADING$ gcc multiexit.c -o multiexit
mg@MG:/mnt/m/training/LINUX/MULTI THREADING$ ./multiexit
Thread 1 exiting with 10
Thread 2 exiting with 20
Thread 3 exiting with 30
Main collected from thread 1: 10
Main collected from thread 2: 20
Main collected from thread 3: 30
Thread 4 exiting with 40
Main collected from thread 4: 40
mg@MG:/mnt/m/training/LINUX/MULTI THREADING$
```